A GraphSAGE MTL Based GNN Model for Joint User Pairing and Power Allocation for Downlink PD-NOMA Systems

Anisha Dwivedi , Shailendra Shukla
220102020 , 220102009

Department of Electronics and Communication Engineering
Indian Institute of Information Technology, Senapati, Manipur, India

Under the supervision of

Dr. Ramesh Chandra Mishra
Assitant Professor, IIIT Manipur

October 26, 2025

Outline

The 5G/6G Challenge

Growing Demands:

- Exponential device growth
- Limited spectrum resources
- High data rate requirements
- Massive IoT connectivity

Traditional Limitations:

- OMA inefficiency
- Poor cell-edge performance

Solution: Non-Orthogonal Multiple Access (NOMA)

Multiple users share same resources through **power-domain multiplexing**.

What is NOMA?

Key Principles:

1. **Power Domain Multiplexing**
   - Near users: Low power
   - Far users: High power
2. **Successive Interference Cancellation (SIC)**
   - Strong users decode weak signals first
   - Cancel interference
   - Decode own signal
3. **Spectral Efficiency Gain**
   - 2-3× capacity improvement
   - Better fairness for cell-edge users

NOMA Signal Model

$$x = \sqrt{\alpha P} s_w + \sqrt{(1-\alpha)P} s_s$$

- s_w: Weak user signal
- s_s: Strong user signal
- α: Power split (0.7-0.9 for weak user)
- P: Total power

Example

500 users $\rightarrow$ 250 pairs

2× spectral efficiency

The User Pairing Challenge

Critical Problem

Which users should be paired together?

Requirements:

- ✓ Sufficient channel gain difference (SIC)
- ✓ Adequate angular separation
- ✓ Maximize sum-rate
- ✓ Ensure fairness
- ✓ Real-time decision

Challenges:

- × Combinatorial complexity: $O(N^3)$
- × 500 users = 10^{1134} combinations!
- × Real-time constraint (¡1 sec)
- × Dynamic channel conditions
- × Multi-objective optimization

Motivation

Can deep learning replace expensive optimization while maintaining near-optimal performance?

System Model

Network Setup:

- Single Base Station (BS)
- N users uniformly distributed
- Downlink transmission
- Bandwidth: 5 MHz per PRB
- Total Power: 46 dBm

Channel Model:

- **3GPP TR 38.901 UMa**
- Path loss + Shadowing + Fading
- Realistic propagation
- 200 scenarios, 500 users each

Received Signal

User i receives:

$$y_i = h_i x + n_i$$

SINR for Weak User

$$\gamma_w = \frac{\alpha P |h_w|^2}{\sigma^2}$$

SINR for Strong User

After SIC:

$$\gamma_s = \frac{(1-\alpha) P |h_s|^2}{\sigma^2}$$

Optimization Problem

Objective: Maximize System Throughput

$$\max_{\mathcal{P},\{\alpha_{ij}\}} \sum_{(i,j)\in\mathcal{P}} R_{ij}$$

where $R_{ij} = R_i + R_j$ is the sum-rate of pair (i, j)

Subject to Constraints:

1. **SIC Feasibility:** $\frac{h_j}{h_i} \geq \gamma_{\text{th}} = 8$ dB $(6.31\times)$
2. **Angular Separation:** $|\theta_i - \theta_j| \geq 25$
3. **Power Allocation:** $0 < \alpha_{ij} < 1$, typically $\alpha \in [0.7, 0.9]$
4. **Matching Constraint:** Each user in at most one pair
5. **Total Power:** $\sum_{(i,j)} P_{ij} \leq P_{\text{total}}$

Complexity

This is an NP-hard combinatorial optimization problem with $O(N^3)$ complexity!

Classical Approaches

1. Static Pairing

- Sort users by channel gain
- Pair strongest with weakest
- Complexity: $O(N \log N)$
- Simple but suboptimal

2. Balanced Pairing

- Divide into strong/weak halves
- Match within respective groups
- Better channel gain balance
- Still greedy approach

3. Bipartite Graph Matching

- Formulate as bipartite graph
- Hungarian algorithm
- Complexity: $O(N^3)$
- Near-optimal solution

4. Blossom Matching (Optimal)

- Maximum-weight matching
- Edmonds' algorithm
- Complexity: $O(N^3)$
- **Optimal benchmark**

Traditional Methods: Performance vs Complexity

Method	Complexity	Throughput	Time (500 users)
Static	$O(N \log N)$	55.6%	5 ms
Balanced	$O(N \log N)$	72.0%	8 ms
Bipartite	$O(N^3)$	100%	35-40 s
Blossom	$O(N^3)$	100%	44.7 s
NOMA-GNN	$O(N \log N)$	**96.5%**	**846 ms**

Key Insight

Trade-off: Fast heuristics are suboptimal, optimal methods are too slow

Our Goal

Achieve near-optimal throughput with fast inference using deep learning

Why Graph Neural Networks?

User Pairing as Graph Problem:

- **Nodes:** Users
- **Edges:** Potential pairs
- **Node Features:** CSI (channel, distance, angle)
- **Edge Labels:** Pair or not pair
- **Edge Attributes:** Sum-rate, power split

Advantages:

- ✓ Natural graph structure
- ✓ Learn from optimal solutions
- ✓ Fast inference
- ✓ Generalization to new scenarios

Graph Construction

Candidate Edge Creation:

- SIC feasible: $h_j/h_i \geq 8$ dB
- Angular separation: ≥ 25
- Strong-weak pairs only

Example

500 users $\rightarrow$ $\sim$3000-5000 candidate edges

Much smaller than $\binom{500}{2} = 124,750$ all pairs!

NOMA-GNN Architecture

GraphSAGE-Based Multi-Task Learning Framework

1. Graph Encoder (GraphSAGE)

- Input: Node features $\mathbf{x}_i = [h_i, d_i, \theta_i, \text{SNR}_i]$
- 3 message-passing layers
- Hidden dimension: 128
- Aggregation: MEAN pooling
- Output: Node embeddings $\mathbf{h}_i \in \mathbb{R}^{128}$

Message Passing

$$\mathbf{h}_i^{(\ell)} = \sigma\left(\mathbf{W}^{(\ell)} \cdot \text{MEAN}\{\mathbf{h}_j^{(\ell-1)}\}\right)$$

2. Multi-Task Decoder

Edge embedding: $\mathbf{e}_{ij} = [\mathbf{h}_i || \mathbf{h}_j]$

Three Prediction Heads:

1. **Pairing Classifier**
   - Binary: Pair or not?
   - Output: $p_{ij} \in [0, 1]$
   - Loss: Binary Cross-Entropy
2. **Sum-Rate Regressor**
   - Predict throughput
   - Output: R_{ij} (bits/s/Hz)
   - Loss: MAE
3. **Power Allocator**
   - Predict α split
   - Output: $\alpha_{ij} \in (0, 1)$
   - Loss: MAE

Training Pipeline

Data Generation:

1. Generate 200 scenarios
2. 500 users per scenario
3. 3GPP UMa channel model
4. Run Blossom for optimal labels
5. Total: 100,000 user samples

Training Strategy:

- **Hard Negative Sampling**
  - Positives: Blossom pairs
  - Negatives: SIC-feasible but not selected
  - Ratio: 1:5 (pos:neg)
- **Feature Normalization**

Multi-Task Loss

$$\begin{aligned}\mathcal{L} =& \lambda_1 \mathcal{L}_{\text{BCE}}(p_{ij}, y_{ij}) \\ &+ \lambda_2 \mathcal{L}_{\text{MAE}}(R_{ij}, \hat{R}_{ij}) \\ &+ \lambda_3 \mathcal{L}_{\text{MAE}}(\alpha_{ij}, \hat{\alpha}_{ij})\end{aligned}$$

Weights: $\lambda_1 = 1.0, \lambda_2 = 0.5, \lambda_3 = 0.3$

Hyperparameters:

- Optimizer: Adam
- Learning rate: 0.001
- Batch size: 32 graphs
- Epochs: 50
- Dropout: 0.3

Inference Pipeline

1. **Input:** New scenario with N users and CSI
2. **Graph Construction:**
   - Apply SIC + angular constraints
   - Create candidate edges
   - Build bipartite message-passing graph
3. **GNN Forward Pass:**
   - Load trained checkpoint
   - Normalize features with saved scaler
   - Predict scores for all candidate edges
4. **Greedy Matching:**
   - Sort edges by predicted sum-rate
   - Greedily select non-conflicting pairs
   - Complexity: $O(E \log E) = O(N^2 \log N)$ worst case
   - Typical: $O(N \log N)$ due to sparse edges
5. **Output:**
   - Selected pairs with predicted α values
   - Estimated total throughput
   - SIC verification report

Channel Model: 3GPP TR 38.901 UMa

Urban Macro Model:

- Carrier: 2.1 GHz
- BS height: 25 m
- UE height: 1.5 m
- Coverage: 500 m radius
- Uniform user distribution

Channel Components:

1. **Path Loss**
   - Distance-dependent
   - LOS/NLOS probability
2. **Shadowing**
   - Log-normal: $\sigma = 7$ dB
3. **Fading**

Path Loss Formula

LOS:

$$\text{PL} = 28 + 22\log_{10}(d) + 20\log_{10}(f_c)$$

NLOS:

$$\text{PL} = 13.54 + 39.08\log_{10}(d) + 20\log_{10}(f_c)$$

Total Channel Gain

$$h = \sqrt{\frac{1}{\text{PL} \times 10^{\text{Shadowing}/10}}} \times \text{Rayleigh}$$

Realistic Modeling:

- Industry standard (3GPP)

Dataset Statistics

Training Data:

- **Scenarios:** 200 (diverse deployments)
- **Users per scenario:** 500
- **Total users:** 100,000
- **Positive pairs:** $\sim$46,400
- **Candidate edges:** $\sim$800,000
- **Features per user:** 7 (CSI-derived)

Test Data:

- **Scenarios:** 1 (held-out)
- **Users:** 500
- **Evaluation:** Throughput, timing

Feature	Range
Distance (m)	10 - 500
Angle (deg)	0 - 360
Path Loss (dB)	80 - 140
Shadowing (dB)	-21 to +21
Fading	0.1 - 3.0
Channel Gain (linear)	10^{-8} - 10^{-4}
SNR (dB)	-10 to 30

Table 1: Feature Statistics

Data Processing:

- CSV format
- PyTorch Geometric graphs
- Z-score normalization
- Saved feature scaler

Implementation Details

Software Stack:

- **Python 3.9+**
- **PyTorch 2.0:** Deep learning
- **PyG 2.3:** Graph neural networks
- **NetworkX:** Graph algorithms
- **NumPy/Pandas:** Data processing

Hardware:

- **Training:**
  - GPU: NVIDIA RTX 3060 (12GB)
  - Time: $\sim$2 hours for 50 epochs
- **Inference:**
  - CPU: Intel i7
  - RAM: 16 GB
  - Time: 846 ms per scenario

Project Structure:

- `config.py` - Parameters
- `data/` - Dataset, normalization
- `models/` - GNN architecture
- `training/` - Training loop, losses
- `inference/` - Fast inference
- `utils/` - Matching, metrics
- `scripts/` - Data prep, verification
- `checkpoints/` - Saved models

Code Organization:

- Modular design
- Well-documented
- End-to-end pipeline

Throughput Performance

Method	Throughput (Gbps)	vs Optimal (%)	Pairs Formed	Time (ms)
Static	40.78	55.6	250	5
Balanced	52.80	72.0	250	8
Bipartite PF	73.30	100.0	232	35,000
Blossom	73.30	100.0	232	44,749
NOMA-GNN	70.74	96.5	225	846

Table 2: Performance Comparison on 500-User Test Scenario

Key Results:

- ✓ **96.5%** of optimal throughput
- ✓ **52.9×** **speedup** over Blossom
- ✓ **73% better** than simple heuristics
- ✓ Real-time inference (¡1 sec)

Trade-off Analysis:

- 3.5% throughput sacrifice
- 98.1% faster computation
- Suitable for edge deployment
- Practical for dense networks

Detailed Performance Metrics

Metric	GNN	Blossom
Throughput (Gbps)	70.74	73.30
Sum-Rate (bits/s/Hz)	15.72	15.80
Spectral Eff. Ratio	99.5%	
Pairs Formed	225	232
Pairing Efficiency	97.0%	
Inference Time	846 ms	44.7 s
Speedup	52.9×	

Table 3: NOMA-GNN vs Optimal

Pairing Quality:

- 225 pairs vs 232 optimal
- 97% pairing efficiency
- All pairs SIC-feasible
- Angular constraints satisfied

Complexity Analysis:

- Training: $O(N^3)$ one-time
- Inference: $O(N \log N)$ per scenario
- Memory: 650 KB model
- Deployment: CPU-friendly

Practical Impact

5G Base Station: Can handle real-time scheduling for 500+ users with near-optimal performance

Model Performance Analysis

Classification Performance:

- **Precision:** $\sim$100%
  - All predicted pairs are valid
- **Recall:** 97.0%
  - Finds 225 out of 232 optimal pairs
- **F1-Score:** 98.5%
  - Excellent balance

Regression Performance:

- Sum-rate prediction accurate
- Power split (α) learned
- Multi-task learning effective

Training Convergence:

- Stable training (50 epochs)
- Generalization to test scenario

Model Characteristics:

- **Parameters:** 166K
- **Size:** 650 KB
- **Layers:** 3 GraphSAGE + 3 MLPs
- **Hidden dim:** 128
- **Dropout:** 0.3

Lightweight

Suitable for edge deployment on resource-constrained base stations

Computational Complexity Comparison

Method	Time Complexity	Space	Real-Time?
Static	$O(N \log N)$	$O(N)$	✓ Yes
Balanced	$O(N \log N)$	$O(N)$	✓ Yes
Bipartite	$O(N^3)$	$O(N^2)$	× No (35s)
Blossom	$O(N^3)$	$O(N^2)$	× No (45s)
NOMA-GNN	$O(N \log N)$	$O(N)$	✓ **Yes (0.85s)**

Table 4: Complexity Analysis for $N = 500$ users

Scalability:

- GNN inference scales well
- Sparse graph structure helps
- Greedy matching is efficient
- Can handle 1000+ users

Practical Deployment:

- **Training:** Offline, one-time
- **Inference:** Online, real-time
- **Updates:** Periodic retraining
- **Hardware:** CPU sufficient

Major Contributions

Realistic Channel Modeling

- 3GPP TR 38.901 UMa standard
- 200 diverse scenarios
- Industry-validated propagation

Comprehensive Baseline Evaluation

- 4 traditional methods implemented
- Thorough performance comparison
- Computational complexity analysis

Novel GNN Framework (NOMA-GNN)

- GraphSAGE-based architecture
- Multi-task learning (pairing + power)
- Physical constraint enforcement
- 96.5% optimal throughput, 52.9× speedup

End-to-End Implementation

- Complete pipeline: data → training → inference
- Lightweight model (166K params, 650 KB)
- Production-ready deployment

Technical Innovations

Graph-Based Reformulation:

- User pairing $\rightarrow$ edge prediction
- Natural problem representation
- Leverages graph structure
- Efficient message passing

Constraint-Aware Design:

- SIC feasibility enforced
- Angular separation considered
- Candidate edge filtering
- Reduces search space

Multi-Task Learning:

- Joint optimization
- Shared representations
- Better generalization
- End-to-end power allocation

Hard Negative Sampling:

- Discriminative learning
- Focuses on difficult cases
- Improves decision boundary
- Better classification

Novel Approach

First work to combine GraphSAGE with NOMA pairing while enforcing physical constraints and achieving near-optimal real-time performance

Key Insights

Finding 1: Learning Beats Heuristics

Data-driven GNN achieves 34-74% better throughput than simple heuristics (Static, Balanced) while maintaining similar computational efficiency

Finding 2: Near-Optimal with Fast Inference

GNN achieves 96.5% of optimal throughput with 52.9$\times$ speedup, making real-time deployment practical for dense networks

Finding 3: Generalization Capability

Model trained on 200 scenarios generalizes well to unseen test scenario, demonstrating learned pairing patterns transfer across deployments

Finding 4: Multi-Task Learning Synergy

Joint learning of pairing decisions, sum-rate prediction, and power allocation leads to better overall performance than separate optimization

Current Limitations

1. **Single-Cell Scenario**
   - Current work focuses on one base station
   - Multi-cell interference not considered
   - Limited to downlink transmission
2. **Perfect CSI Assumption**
   - Assumes accurate channel knowledge
   - Channel estimation errors not modeled
   - Feedback delay not considered
3. **Static User Distribution**
   - Users assumed stationary during scheduling
   - Mobility not incorporated
   - Doppler effects ignored
4. **Two-User Clustering Only**
   - Pairs limited to 2 users
   - No multi-user NOMA (3+ users per PRB)
   - Scalability to larger clusters unexplored

Future Research Directions

Near-Term Extensions:

1. **Multi-Cell Coordination**
   - Inter-cell interference
   - Coordinated scheduling
   - Graph attention mechanisms
2. **Channel Uncertainty**
   - Imperfect CSI modeling
   - Robust pairing strategies
   - Estimation error tolerance
3. **Dynamic Scenarios**
   - User mobility
   - Time-varying channels
   - Recurrent GNN architectures
4. **Multi-User Clustering**
   - 3+ users per resource
   - Hypergraph neural networks
   - Hierarchical clustering

Long-Term Vision:

5. **Reinforcement Learning**
   - Online learning
   - Adaptive policies
   - Reward shaping
6. **Federated Learning**
   - Distributed training
   - Privacy preservation
   - Multi-operator collaboration
7. **Hardware Deployment**
   - FPGA/ASIC implementation
   - Model compression
   - Quantization
8. **6G Integration**
   - THz communications
   - Massive MIMO-NOMA
   - AI-native networks

Summary

Problem Addressed:

- User pairing in NOMA systems is computationally expensive ($O(N^3)$)
- Traditional heuristics are fast but suboptimal
- Optimal methods too slow for real-time deployment

Our Solution: NOMA-GNN

- Graph neural network framework based on GraphSAGE
- Learns from optimal solutions (Blossom matching)
- Multi-task learning: pairing + power allocation
- Enforces physical constraints (SIC, angular separation)

Achievements:

- ✓ **96.5%** of optimal throughput (70.74 vs 73.30 Gbps)
- ✓ **52.9× speedup** (846 ms vs 44.7 s)
- ✓ **Real-time inference** for 500 users
- ✓ **Lightweight model** (166K params, 650 KB)
- ✓ Practical deployment for dense 5G/6G networks

Impact and Significance

Theoretical Contributions:

- Novel graph formulation
- Multi-task learning framework
- Complexity reduction proof
- Generalization analysis

Practical Impact:

- Real-time NOMA deployment
- Edge base station suitable
- Scalable to dense networks
- Industry-standard channels

Broader Applications:

- 5G NR networks
- 6G research
- IoT massive connectivity
- Smart city deployments
- Industrial wireless

Research Enabler:

- Open-source implementation
- Reproducible experiments
- Extensible framework
- Foundation for future work

Bottom Line

Deep learning can effectively replace expensive optimization in NOMA systems while

Publications and Deliverables

Project Outputs:

1. **College Project Report**
   - 55-60 pages comprehensive documentation
   - 11 main chapters + 3 appendices
   - 30+ references, 50+ tables, 100+ equations
2. **IEEE Conference Paper (Under Preparation)**
   - 10-page submission format
   - Complete experimental evaluation
   - Comparative analysis with baselines
3. **Complete Implementation**
   - End-to-end Python pipeline
   - Well-documented modular code
   - Trained models and checkpoints
   - Ready for deployment
4. **Dataset**
   - 200 realistic scenarios (3GPP UMa)
   - 100,000 user samples
   - Optimal pairing labels
   - Public availability planned

Graph Neural Network-Based User Pairing and Power Allocation for Downlink NOMA Systems

Anisha Dwivedi, Shailendra Shukla, and Ramesh Ch. Mishra
Department of Electronics and Communication Engineering
Indian Institute of Information Technology, Senapati, Manipur

Questions?

Contact: {anishas1121, shailendra.iiitsm}@gmail.com

Backup: Technical Details

GNN Architecture Details:

- **Encoder:** 3-layer GraphSAGE
  - Input: 4D features $\rightarrow$ 128D embeddings
  - Aggregation: MEAN pooling
  - Activation: ReLU
- **Decoders:** 3 separate MLPs
  - Classifier: 256$\rightarrow$128$\rightarrow$64$\rightarrow$1 (sigmoid)
  - Sum-rate: 256$\rightarrow$128$\rightarrow$64$\rightarrow$1 (ReLU)
  - Power: 256$\rightarrow$128$\rightarrow$64$\rightarrow$1 (sigmoid)
- **Training:**
  - Loss: $\mathcal{L} = 1.0 \cdot \mathcal{L}_{\text{BCE}} + 0.5 \cdot \mathcal{L}_{\text{rate}} + 0.3 \cdot \mathcal{L}_{\text{power}}$
  - Optimizer: Adam (lr=0.001)
  - Batch: 32 graphs
  - Epochs: 50

Backup: Complexity Derivation

NOMA-GNN Inference Complexity:

1. **Graph Construction:** $O(N^2)$ worst case
   - Check all pairs for SIC + angular constraints
   - Typically $O(N)$ due to locality
2. **GNN Forward Pass:** $O(E \cdot D^2)$
   - $E \approx$ 3000-5000 edges (sparse)
   - $D = 128$ hidden dimension
   - Effectively $O(N)$ for sparse graphs
3. **Greedy Matching:** $O(E \log E)$
   - Sort edges by score: $O(E \log E)$
   - Select non-conflicting: $O(E)$
   - Total: $O(E \log E) = O(N \log N)$ typical

Overall: $O(N \log N)$ **typical,** $O(N^2 \log N)$ **worst case**

Compare to Blossom: $O(N^3)$ always